

50 AI/LLM Engineering Interview Questions

You should know before your next AI Engineer interview.
Not just definitions.

TOPICS COVERED:

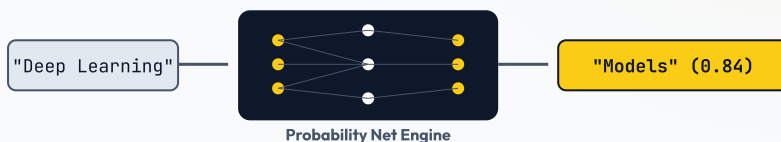
LLM fundamentals — Inference — Context — RAG — Fine-tuning — Evaluation — Production

50 Questions. 50 Answers. Cheat Sheet →

What is an LLM?

An LLM is a neural network trained on large amounts of text to model the probability of token sequences.

During generation, it predicts the next token based on the tokens already provided.



INTERVIEW POINT

An LLM is not a database that retrieves stored answers. It generates outputs from learned statistical representations.

What is a **token**?

A token is a unit of text processed by an LLM. It may represent:

- A complete word
- Part of a word
- Punctuation
- Whitespace or special characters



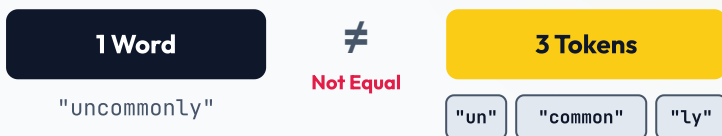
WHY IT MATTERS

Token count affects context limits, latency and API cost.

Token vs Word: The Difference

They are not the same. One word can split into multiple tokens, especially uncommon vocabulary.

Therefore, 1,000 words $\neq$ necessarily 1,000 tokens.

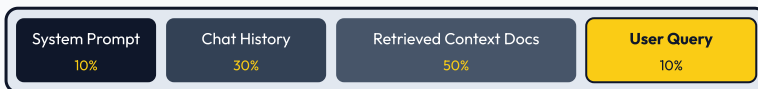


PRODUCTION RULE

Always measure token usage in production LLM applications, not word or character counts, to budget costs reliably.

What is the **context window**?

The context window is the maximum number of tokens a model can consider for a single inference request.



Context Window Boundary (Max Limit)

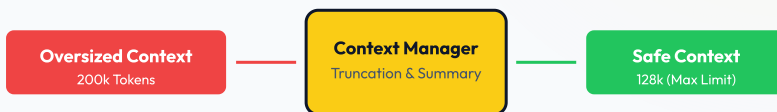
INTERVIEW INSIGHT

A larger context window does not automatically guarantee higher quality answers. Context still needs to be clean, relevant, and well-structured.

Context Window Exceeded?

When the context window is exceeded, standard APIs reject the request with an error.

Production systems must manage this dynamically using summarizers, truncation filters, or vector retrievers.



GOOD ANSWER

"Don't simply keep adding context. Control what information enters the model."

What is a **Transformer**?

A Transformer is a neural network architecture based primarily on attention mechanisms. It processes token sequences in parallel.



CORE CONCEPT

Transformers became the foundation for modern LLMs by allowing massive parallel training on GPUs.

What is **self-attention** ?

Self-attention allows each token to determine which other tokens in the sequence are important for representing its meaning.



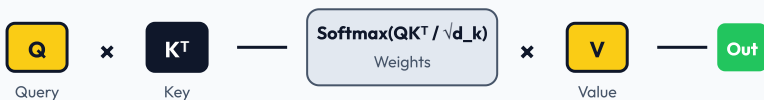
CONCEPT VISUALIZED

The query Projects dynamic weights, creating contextually accurate semantic representations of text token paths.

What are **Query, Key, Value** ?

Self-attention projects each token embedding into three vectors:

- **Query (Q)**: What details am I searching for?
- **Key (K)**: What details do I contain?
- **Value (V)**: What content do I pass forward?



EQUATION

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

Why divided by $\sqrt{d_k}$?

Without scaling, the dot products of high-dimensional vectors Q and K grow very large in magnitude.

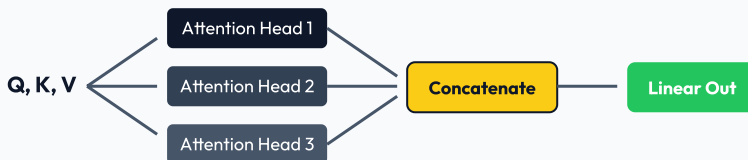
Large values push the softmax function into flat, saturated regions where gradients become extremely small, slowing or halting training.

MATHEMATICAL REASON

Dividing by $\sqrt{d_k}$ (the standard key vector dimension) stabilizes the variance of the dot product to 1. This prevents the vanishing gradient problem during backpropagation.

What is **multi-head attention** ?

Instead of calculating a single attention score, the model splits Q, K, and V into multiple projection heads that run in parallel.

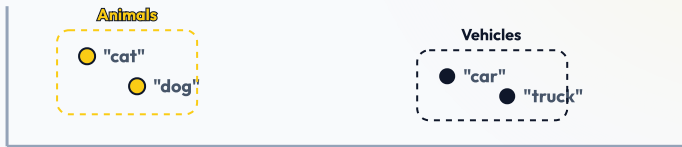


INTERVIEW INSIGHT

Multi-head attention gives the model multiple independent representation subspaces to capture syntax, entity links, and context simultaneously.

What is an embedding?

An embedding is a dense numerical representation of data in a high-dimensional vector space.



INTERVIEW CONCEPT

Embeddings project raw text into vector coordinates where semantic associations are represented as spatial proximity (geometry).

Embeddings vs Generated Text

Embeddings and generated text serve entirely different operational goals:

- **Embeddings:** Project semantic meaning into fixed vectors. Perfect for search, semantic retrieval, and clustering.
- **LLMs:** Generate conversational token sequences sequentially. Perfect for reasoning, summarization, and writing.

SUMMARY DISTINCTION

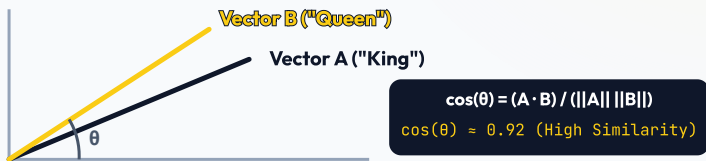
Embeddings = Representation & Similarity Search.

LLMs = Reasoning & Text Generation.

What is cosine similarity?

It measures the cosine of the angle between two vectors in a high-dimensional space.

A value of 1 means they point in the exact same direction; 0 means orthogonal; -1 means opposite.



USE CASE

Widely utilized in vector databases for semantic document search and chunk retrieval pipelines.

Why use embeddings in RAG?

Instead of matching documents solely using keyword matching (TF-IDF/BM25), embeddings allow retrieval based on semantic intent.

For example, a user search query for "How long can employees work remotely?" will successfully match a policy document stating "Remote work is permitted for up to 30 days," even with zero exact word overlap.

RAG FOUNDATION

Semantic retrieval matches concepts over keywords, returning high-context relevance to the generation LLM.

What makes a **good** embedding model?

Selecting an embedding model requires checking multiple parameters:

- ✓ Domain alignment (e.g. medical, code)
- ✓ Multi-lingual capability
- ✓ Dimension length (affects database index size & latency)
- ✓ Maximum token input limit
- ✓ MTEB benchmark performance

INTERVIEW TRAP

The largest model is not automatically the best. A fast, medium-dimension model is often superior for low-latency searches.

What is **inference**?

Inference is using a trained model to process inputs and generate tokens.

For an autoregressive LLM, this involves iteratively feeding predicted tokens back into the input sequence for the next prediction.

OPERATIONAL FOCUS

While training is a compute-heavy batch process, inference is memory-bandwidth bound, focusing on latency per token generation.

What is **autoregressive** generation?

The model generates tokens one by one sequentially. The generated output of step t becomes part of the input context for step $t+1$.



CONSEQUENCE

Inference speed declines as output sequences lengthen, requiring techniques like KV caching to maintain speeds.

What is **greedy decoding**?

Greedy decoding selects the token with the highest absolute probability at each prediction step.

If probabilities are: $P("A") = 0.60$, $P("B") = 0.25$, $P("C") = 0.15$, greedy decoding will output "A".

✓ **Pros:** Fast, simple, and deterministic.

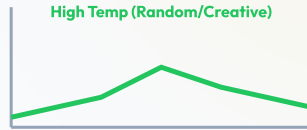
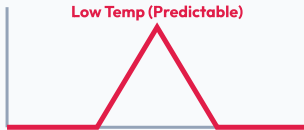
✗ **Cons:** Can result in repetitive loops or suboptimal sequences.

CORE LESSON

Locally optimal choices (highest probability at current step) can miss globally optimal sentences.

What is **temperature**?

Temperature modifies the probability distribution used during sampling:



IMPORTANT

Temperature does not make the model "more intelligent." It simply scales prediction randomness.

What are **Top-K** and **Top-P**?

These techniques filter the set of candidate tokens before sampling:

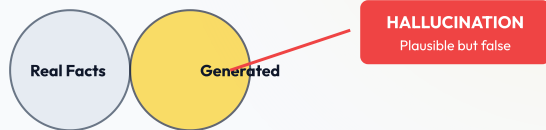


PRODUCTION TIP

Both parameters are decoding configuration options. They filter out low-probability tail tokens to keep the text coherent.

Why do LLMs hallucinate?

LLMs generate likely continuations rather than validating facts.



CORE CONCEPT

An LLM is optimized for syntax likelihood, not truth. Conflicting data, weak contexts, or overgeneralization trigger hallucinations.

Does RAG **eliminate** it?

No. RAG reduces hallucinations by providing relevant external context.

But the model can still retrieve the wrong document, ignore relevant evidence, misinterpret retrieved text, combine unrelated sources, or generate unsupported claims.

KEY INTERVIEW LINE

RAG reduces hallucinations; it does not eliminate them.

What is **grounding**?

Grounding means constraining an AI response using reliable external information.

Examples: retrieved documents, databases, APIs, search results, and enterprise knowledge bases.

GROUNDING PRINCIPLE

A grounded response should be traceable to the factual information supplied to the model, not generated from internal weights.

How to **reduce** hallucinations?

A strong engineering solution requires multiple system layers:

- ✓ Better retrieval quality
- ✓ Better prompt formatting and explicit constraints
- ✓ Structured outputs (JSON validation)
- ✓ Citation and evidence cross-validation

SYSTEM DESIGN POINT

There is no single "anti-hallucination prompt." You must build guardrails around your model outputs.

How to **detect** hallucinations?

Run automated checks on generation outputs to measure:

- Factual correctness against known references
- Faithfulness to the provided input context
- Citation validity — checking for fake references

DETECTION STACK

Combine LLM-as-a-judge, deterministic parsing rules, and human reviews to isolate hallucinated content.

What is **prompt engineering**?

Prompt engineering is styling inputs, instructions, and examples to guide LLM behavior without training weights.

It includes defining clear roles, explicit constraints, output formats, few-shot examples, and logical workflows.

CORE VALUE

Prompt engineering is the quickest way to prototype AI behavior, serving as a baseline before code optimizations or model tuning.

What is **few-shot prompting**?

Few-shot prompting feeds sample input-output demonstrations directly into the prompt context.

This allows the model to learn format standards, output styles, and logical parsing steps on the fly before generating final responses.

CONCEPT VISUAL

INPUT:

Paris —>

OUTPUT:

France

INPUT:

Tokyo —>

OUTPUT:

Japan

INPUT:

Berlin —>

OUTPUT:

[Model outputs Germany]

Prompt vs Context Engineering

These terms denote different levels of input scope control:

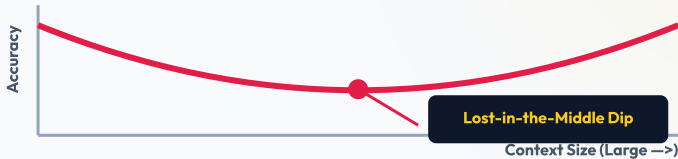
- **Prompt Engineering:** Optimizes core text instructions and format rules.
- **Context Engineering:** Designs the complete input state dynamically, including retrieval databases, short-term history, active memory, and tools.

SYSTEM MINDSET

Modern agentic systems rely heavily on context engineering to manage data states dynamically throughout user sessions.

Why can more context **degrade** output?

Large contexts introduce noise and cause the model to overlook central data.



GOLDEN RULE

Maximize context relevance, not token count. Excess data dilutes instruction weight.

What is structured output ?

Structured output forces the model to respond in a predictable data format, such as a validated JSON schema:

```
{  
  "candidate_name": "Devadharshani",  
  "skills": ["RAG", "LLMs", "Agents"],  
  "experience_years": 3  
}
```

BENEFITS

Guarantees reliable downstream parsing, prevents API processing crashes, and enables robust type validations.

What is RAG ?

Retrieval-Augmented Generation couples external database search with LLM text generation:



FLOW SUMMARY

1. Query is sent to DB. — 2. Relevant document blocks are retrieved. — 3. Text blocks are combined into the prompt. — 4. LLM outputs a grounded response.

RAG vs Fine-tuning: When RAG?

Use RAG when your application requires access to:

- Frequently updated data (e.g. news, stocks)
- Private enterprise files or internal logs
- Verifiable source document links & citations

RAG

Dynamic & Private Knowledge

Injects External Data

Updates Model Parameters

Fine-Tuning

Static Behavior & Style

DIFFERENCE

RAG changes the external facts visible to the model. Fine-tuning adjusts the internal behavior pattern of the model.

When to use **fine-tuning**?

Fine-tuning is ideal when you need to change how the model behaves or specializes for specific workflows:



Adapts style and formatting behaviors

INTERVIEW INSIGHT

Fine-tuning specializes model formatting and task behavior, but is not a reliable way to store factual knowledge bases.

RAG or FT for **policy files**?

Start with **RAG** to answer questions from corporate policy documents.

Why?

- Policy guidelines update frequently
- RAG allows document indices to update instantly
- RAG supplies precise page number citations for audits

TRADE-OFF

Fine-tuning is expensive, lacks citation logs, and requires a full model retraining cycle whenever policies change.

Can FT teach **new facts**?

While fine-tuning can inject features of training data into weights, it is prone to severe hallucinations when queried on edge cases.

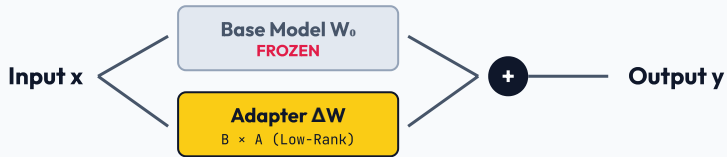
Relying on fine-tuning weights as a dynamic search database is a fragile system design pattern.

SYSTEM STANDARD

For factual query answering, retrieval-augmented pipelines are significantly more reliable and cheaper than model fine-tuning.

What is LoRA?

LoRA (Low-Rank Adaptation) freezes original base model parameters and injects trainable low-rank decomposition matrices in parallel.



BENEFIT

Reduces trainable parameters by 99%, lowering GPU memory footprints for fine-tuning cycles.

What is QLoRA?

QLoRA combines model quantization with LoRA adapters:

- The massive base LLM is compressed into a highly compact 4-bit precision format.
- The LoRA adapter matrices remain in high-fidelity 16-bit precision for parameter updates.

EFFICIENCY GAIN

Enables developers to fine-tune massive models on consumer-grade hardware instead of expensive cloud GPU clusters.

What is quantization?

Quantization compresses weights by reducing numerical precision (FP32 $\rightarrow$ INT8 $\rightarrow$ INT4).



TRADE-OFF

Shrinks memory size and speeds up generation, but can trigger slight accuracy degradations.

Optimization vs Fine-tuning

These techniques optimize different stages of the model lifecycle:

- **Fine-Tuning:** Modifies internal parameters or adapter weights to improve logic accuracy and style formatting.
- **Inference Optimization:** Accelerates text generation efficiency without altering model logic (e.g. KV cache, quantization, flash attention).

PRODUCTION RULE

Always implement inference optimizations before scaling production serving infrastructures to lower costs.

What is a KV cache?

KV cache stores Key and Value attention tensors of previous tokens in memory during sequential autoregressive generation.



Avoids redundant $O(N)$ calculations per token!

WHY IT MATTERS

Prevents redundant matrix operations over older history, converting latency growth from quadratic to linear.

Why is evaluation **difficult** ?

Traditional code runs on deterministic assertions. Natural language prompts produce diverse, open-ended responses.

Evaluating an LLM application requires measuring multiple quality dimensions: factual correctness, query relevance, style constraints, safety guidelines, and user satisfaction.

MODERN STANDARD

Deterministic checks (e.g. regex) must be combined with LLM judges and human audits to evaluate quality effectively.

What is **LLM-as-a-Judge**?

This technique uses an independent, highly capable LLM (like GPT-4) to grade student model answers against explicit quality rubrics.

Workflow: query input → student generated answer → judge evaluation prompt → numerical grade score + feedback logic.

CRITICAL CAVEAT

LLM judges are prone to systematic position and length biases. Never treat judge ratings as absolute ground truths.

What is a **golden dataset**?

A golden dataset is a curated, high-quality suite of testing samples containing representative user queries, target reference data, and expected answers.

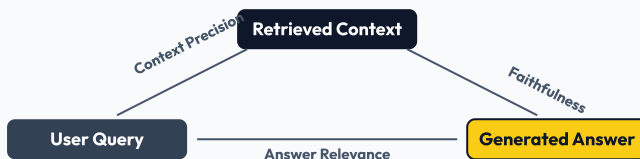
It acts as a regression test suite, enabling engineers to measure how prompt changes or model swaps affect output quality.

SYSTEM BENEFIT

Prevents "vibe-check" debugging. Provides data-driven metrics to verify whether system updates improve or break performance.

What metrics to track in RAG?

Evaluate retrieval accuracy and faithfulness using RAG metrics:



INTERVIEW INSIGHT

Don't evaluate only the final answer. Benchmark the retrieval pipeline components separately to isolate bugs.

Why is **accuracy** not enough?

An application can produce highly accurate answers while failing other production requirements:

- Latency is too high for interactive chat
- API token costs exceed query value budgets
- Responses leak sensitive system prompts
- Outputs lack verifiable source citations

CORE LESSON

Production AI evaluation requires multi-dimensional analysis: latency, cost, safety, grounding, and format correctness.

How to reduce **latency**?

Optimize latency across the application lifecycle:

- **Stream Outputs:** Display tokens to users as they generate.
- **Prune Prompts:** Remove redundant context tokens.
- **Model Routing:** Route simple queries to smaller, faster LLMs.
- **Caching:** Store previous responses for repeating user queries.
- **Parallelization:** Execute tool calls and DB queries in parallel.

INTERVIEW PRINCIPLE

Measure latency sources first. Do not blindly optimize code without identifying the true system bottleneck.

How to reduce LLM **costs**?

Cost is driven by: Input tokens + Output tokens + Model call counts. Optimize via:

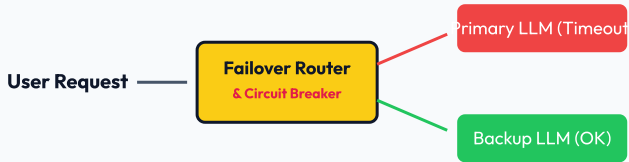
- **Model Tiering:** Route simple classification tasks to cheaper micro models.
- **Context Compression:** Summarize long history logs dynamically.
- **Semantic Caching:** Avoid calling the LLM for repeating intents.
- **Batching:** Use offline batch APIs for non-realtime operations.

SYSTEM GUIDELINE

Cost reduction is a system architecture challenge, not just a model choice decision.

What if the LLM API fails?

Production AI systems must handle API outages gracefully using failover mechanisms:



INTERVIEW AXIOM

"Never assume an API endpoint is 100% available. Implement circuit breakers, fallbacks, and retry budgets."

Design a production **AI app**

A reliable production system separates model serving from business logic:

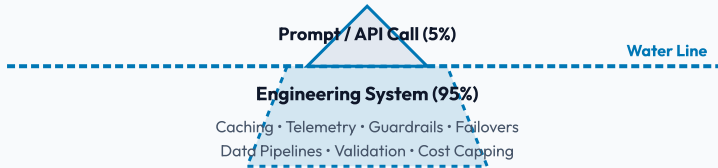


SIDECARS NEEDED

Production systems require telemetry logging, prompt tracking, execution metrics, cost budgets, and evaluation pipelines.

What is the biggest mistake?

Treating the LLM as the entire system.



FINAL INSIGHT

Building a basic prototype requires a prompt. Designing a production system requires comprehensive software engineering practices.

10 Concepts You **MUST** Master

1. Tokens

How text gets parsed into model inputs.

2. Context Window

Active memory limits of model runs.

3. Attention

How tokens weigh semantic relationships.

4. Embeddings

Representing concepts as vectors.

5. Inference

Token generation mechanics.

6. Decoding

Token sampling parameters (Top-K/Top-P).

7. RAG

Retrieving database contexts dynamically.

8. Fine-Tuning

Adapting styling formats & behaviors.

9. Evaluation

Measuring quality beyond simple accuracy.

10. Production

Scaling cost, latency, & availability.

✓ Solidify these foundations for your engineering loop.

Don't Memorize. Understand the system.

PRODUCTION (Latency, Cost, Scalability, Security)

RELIABILITY (Evaluation, Guardrails, Observability)

APPLICATION (Prompt, Context, RAG, Tools)

MODEL (Tokens, Attention, Embeddings)

Separating **"I have called an API"** from **"I can engineer a reliable AI system."**

 Day 1 Complete

 Save this series

 **Next:** 40 RAG Interview Questions